

Project 2B — Complete Build Guide

Customer Analytics & Churn Dashboard with A/B Testing Simulation

This guide takes you from zero to a finished, resume-ready, interview-defensible project. Follow it phase by phase — don't skip the SQL phase to jump to modeling, since the SQL depth is exactly what makes this project different from a generic "churn prediction" notebook that everyone else submits.

0. What you need to know before starting (and where to learn it fast)

You don't need to master everything upfront — learn just-in-time, phase by phase. But here's the minimum baseline and where to get it quickly:

Skill	Minimum level needed	Fast resource
SQL	SELECT, JOIN, GROUP BY comfortable; window functions/CTEs can be learned during the project	Mode SQL Tutorial (free, excellent, has a real query runner)
Python (pandas)	Comfortable reading/filtering dataframes	You likely already have this from coursework
Basic statistics	Mean/std, what a p-value means, t-test intuition	Khan Academy "Hypothesis testing" module (a few hours) — don't skip this, it's the conceptual core of the A/B test part
Git/GitHub	Init repo, commit, push, write a README	If new to this, 30 min tutorial is enough

You do **not** need deep ML knowledge for this project — the model here is intentionally simple (Track 2B is about analysis + SQL + statistical reasoning, not sophisticated modeling; that's what Project 2A is for).

1. Tools & Platform Setup

Everything here is free. Install/set up in this order:

1. **PostgreSQL** — the database you'll run all your SQL analysis against.

- Easiest path: install Postgres.app (Mac) or the official installer (Windows/Linux), or even easier — use **Supabase** (free tier, gives you a hosted Postgres instance with a web SQL editor, zero local setup). For a laptop-based student project, I'd actually recommend Supabase — one less thing to configure.
2. **DBeaver** (free, cross-platform) or **pgAdmin** — a GUI to browse tables and run/save queries comfortably. DBeaver is more pleasant to use.
 3. **Python environment** — use **Jupyter Notebook** (via Anaconda or plain `pip install jupyter`) for the EDA/modeling phase. Libraries you'll need:

```
pip install pandas numpy scikit-learn scipy matplotlib seaborn sqlalchemy  
psycopg2-binary statsmodels
```

4. **Dashboard tool** — pick one:
 - **Tableau Public** (free, industry-standard, what banks actually use — recommended if you want the resume line "Tableau")
 - OR **Power BI Desktop** (free, Windows only, also very common in banking/finance analytics teams)
 - I'd lean **Tableau Public** since it's cross-platform and the portfolio link is easily shareable (Tableau Public hosts your dashboard on a public URL you can put in your resume).
5. **GitHub repo** — create it now, before you write any code. Commit early and often; a commit history that shows the project evolving over weeks (not one big dump) reads better to anyone who checks.

2. Dataset

Use "**Churn for Bank Customers**" on Kaggle:

<https://www.kaggle.com/datasets/mathchi/churn-for-bank-customers>

Why this one specifically: it's a clean, well-known 10,000-row banking dataset with exactly the columns you want for a finance-domain story (directly relevant to Citi/HSBC/BNY):

`CustomerId, Surname, CreditScore, Geography, Gender, Age, Tenure, Balance,
NumOfProducts, HasCrCard, IsActiveMember, EstimatedSalary, Exited`

`Exited` (1 = churned, 0 = retained) is your target variable. It's small enough to be fast to iterate on, but has enough real-world messiness (imbalanced classes, mixed categorical/numeric features) to make the analysis genuinely interesting.

Download the CSV, then load it into Postgres (Phase 1 below).

3. Phase-by-Phase Build

Phase 1 — Load & Clean Data in PostgreSQL (Week 1)

1. Create a database `churn_analytics`.
2. Create a `customers` table matching the CSV schema, then load the CSV in (DBeaver has a GUI import wizard; or use Python/SQLAlchemy: `df.to_sql('customers', engine, if_exists='replace')`)).
3. Run basic data-quality checks in **SQL, not pandas** — this is deliberate, since SQL data-quality auditing is exactly what analyst interviews probe:

sql

```
-- check for nulls
SELECT COUNT(*) FILTER (WHERE "CreditScore" IS NULL) AS null_credit_score,
       COUNT(*) FILTER (WHERE "Age" IS NULL) AS null_age
FROM customers;

-- check for duplicate customers
SELECT "CustomerId", COUNT(*) FROM customers GROUP BY "CustomerId" HAVING CO

-- sanity range checks
SELECT MIN("Age"), MAX("Age"), MIN("Balance"), MAX("Balance") FROM customers
```

4. Document what you found and fixed (even if the answer is "dataset was clean") — this becomes a short "Data Quality" section in your README.

Phase 2 — SQL Exploratory Analysis (Week 1-2)

This is the heart of the project — spend real time here. Write these as **saved, commented .sql files** in your repo (`/sql/01_churn_by_segment.sql` etc.), not one-off scratch queries — a recruiter or interviewer skimming your GitHub should be able to open the `/sql` folder and immediately see analytical depth.

Queries to write (each teaches you a different SQL technique):

a) Overall churn rate

sql

```
SELECT
  ROUND(100.0 * SUM(CASE WHEN "Exited"=1 THEN 1 ELSE 0 END) / COUNT(*), 2) AS churn_rate_pct
FROM customers;
```

b) Churn by geography and gender (GROUP BY + aggregation)

sql

```
SELECT "Geography", "Gender",
  COUNT(*) AS total_customers,
  SUM("Exited") AS churned,
  ROUND(100.0 * SUM("Exited") / COUNT(*), 2) AS churn_rate_pct
FROM customers
GROUP BY "Geography", "Gender"
ORDER BY churn_rate_pct DESC;
```

c) Churn by age bucket (CASE WHEN bucketing — a very commonly asked SQL interview pattern)

sql

```
SELECT
  CASE
    WHEN "Age" < 30 THEN '<30'
    WHEN "Age" BETWEEN 30 AND 40 THEN '30-40'
    WHEN "Age" BETWEEN 41 AND 50 THEN '41-50'
    ELSE '50+'
  END AS age_bucket,
  COUNT(*) AS total, SUM("Exited") AS churned,
  ROUND(100.0 * SUM("Exited") / COUNT(*), 2) AS churn_rate_pct
FROM customers
GROUP BY age_bucket
ORDER BY age_bucket;
```

d) Product holding vs churn (a classic finance insight — customers with only 1 product often churn more)

sql

```
SELECT "NumOfProducts",
       COUNT(*) AS total, SUM("Exited") AS churned,
       ROUND(100.0*SUM("Exited")/COUNT(*),2) AS churn_rate_pct
FROM customers
GROUP BY "NumOfProducts"
ORDER BY "NumOfProducts";
```

e) Window function — rank countries by churn rate (demonstrates window functions, a common interview gap for students)

```
sql

SELECT "Geography", churn_rate_pct,
       RANK() OVER (ORDER BY churn_rate_pct DESC) AS churn_rank
FROM (
  SELECT "Geography", ROUND(100.0*SUM("Exited")/COUNT(*),2) AS churn_rate_pct
  FROM customers GROUP BY "Geography"
) t;
```

f) CTE — "high value at-risk customers" (combine balance + activity + churn signal — this is the kind of business-relevant query a bank analyst is actually asked for)

```
sql

WITH risk_flagged AS (
  SELECT *,
         CASE WHEN "IsActiveMember" = 0 AND "Balance" > 100000 THEN 1 ELSE 0 END AS
  FROM customers
)
SELECT COUNT(*) AS at_risk_customers, SUM("Balance") AS total_balance_at_risk
FROM risk_flagged
WHERE high_value_at_risk = 1;
```

Write 8–10 queries like this covering: aggregation, CASE bucketing, window functions, CTEs, subqueries, and at least one query with a HAVING clause. Save all outputs as a short markdown findings doc — this doc becomes the backbone of your "executive summary" later.

Phase 3 — Python EDA & Visualization (Week 2)

Pull data from Postgres into pandas (via SQLAlchemy) rather than re-reading the CSV — this reinforces the "SQL as source of truth" narrative and lets you show a query-to-dataframe pipeline in your notebook.

```
python
```

```
from sqlalchemy import create_engine
import pandas as pd

engine = create_engine("postgresql://user:pass@localhost:5432/churn_analytics")
df = pd.read_sql("SELECT * FROM customers", engine)
```

Build 4-6 visualizations with seaborn/matplotlib that mirror your SQL findings visually (churn by geography, age distribution split by churn, correlation heatmap of numeric features, balance distribution by churn status). Keep this notebook focused on *exploration*, not modeling.

Phase 4 — Churn Prediction Model (Week 3)

Keep this intentionally lighter than Project 2A's model — the point of 2B is the *analysis*, not modeling sophistication.

1. Feature engineering: encode `Geography`/`Gender`, scale numeric features.
2. Train a **Logistic Regression** as your interpretable baseline, and a **Random Forest** as your stronger model.
3. Evaluate with accuracy, precision/recall, F1, and ROC-AUC — and explicitly discuss class imbalance (churn is usually ~20% of the dataset) and why accuracy alone is misleading here.
4. Pull out feature importances (Random Forest) or coefficients (Logistic Regression) and connect them back to your SQL findings — e.g., "SQL analysis showed 1-product customers churn more; feature importance confirms `NumOfProducts` is the top predictor." This connection between SQL insight and model output is a strong narrative for interviews.

```
python
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, roc_auc_score
```

Phase 5 — A/B Testing Simulation (Week 3-4) — the differentiating part

This dataset has no real campaign data, so you'll **simulate** a retention campaign — that's fine and completely standard practice for a learning project, as long as you're upfront about it being simulated (say so explicitly in your README; don't imply it's real experimental data).

The design:

1. **Define the intervention:** "The bank offers a fee waiver + relationship manager call to at-risk customers." At-risk = customers your model scored high on churn probability.
2. **Random assignment:** Take your at-risk customer segment and randomly split into **Control** (no intervention) and **Treatment** (gets the offer), 50/50, using a fixed random seed for reproducibility.

python

```
at_risk = df[df['predicted_churn_prob'] > 0.5].copy()
at_risk['group'] = np.random.choice(['control', 'treatment'], size=len(at_risk))
```

3. **Simulate an effect (since we don't have a real outcome):** assume the treatment genuinely reduces churn probability by a plausible amount (e.g., a 5-percentage-point reduction), and simulate actual churn outcomes accordingly:

python

```
# baseline churn from model probability, minus a treatment effect
at_risk['sim_churn_prob'] = np.where(
    at_risk['group'] == 'treatment',
    (at_risk['predicted_churn_prob'] - 0.05).clip(lower=0),
    at_risk['predicted_churn_prob']
)
at_risk['simulated_churned'] = np.random.binomial(1, at_risk['sim_churn_prob'])
```

4. **Analyze the result like a real A/B test:**
 - Compute churn rate in control vs treatment
 - Run a **two-proportion z-test** (or chi-square test of independence) to check statistical significance:

python

```

from statsmodels.stats.proportion import proportions_ztest
control_churned = at_risk[at_risk.group=='control'].simulated_churned.sum()
treat_churned = at_risk[at_risk.group=='treatment'].simulated_churned.sum()
n_control = (at_risk.group=='control').sum()
n_treat = (at_risk.group=='treatment').sum()

stat, pval = proportions_ztest([treat_churned, control_churned], [n_treat, n_control])
print(f"z-stat: {stat:.3f}, p-value: {pval:.4f}")

```

- Report the **effect size**, confidence interval, and p-value — not just "significant/not significant." Interviewers specifically probe whether you understand p-values are not the whole story.
5. **Bonus (shows real depth):** do a quick **power analysis** — given your sample size, what's the minimum detectable effect? This is something almost no student-level project includes and is a genuine differentiator if you can explain it in an interview.

python

```

from statsmodels.stats.power import NormalIndPower
analysis = NormalIndPower()
# solve for effect size given n, alpha, power

```

Be ready to explain in an interview: why you randomized, why you used a z-test vs t-test vs chi-square (they're closely related for proportions — know the distinction), what a Type I/Type II error would mean in this business context (wasting retention budget vs losing a customer you could have saved).

Phase 6 — Dashboard (Week 4)

Build a dashboard in Tableau Public with at least these views:

1. Overall churn KPI tile + trend by segment
2. Churn rate by geography/age/product-count (matches your SQL queries)
3. A/B test results panel — control vs treatment churn rate with the p-value annotated
4. A "high-value at-risk customers" table (from your CTE query) — this is the page a retention team would actually use

Publish it to Tableau Public and get a shareable link — put this link directly on your resume/LinkedIn next to the project bullet.

Phase 7 — Executive Summary (Week 4)

Write a **one-page** summary, business-audience style (no code, no jargon dump), structured as:

- **Headline finding** (e.g., "Customers with only 1 product churn at 2.3x the rate of multi-product customers")
- **3-4 supporting findings** from your SQL/EDA
- **The A/B test result and recommendation** ("Simulated retention campaign showed a statistically significant X pp reduction in churn (p=0.0Y); recommend piloting on the 1-product, high-balance segment")
- **Caveat section** — explicitly state the A/B results are simulated for demonstration, and what you'd need (real experiment) to validate before rolling out

This document matters more than people expect — banks specifically test written business communication in analyst interviews, and having a polished one-pager to reference (or literally hand over) is a strong signal.

Phase 8 — GitHub Polish

Structure your repo:

```
/sql           - all your saved, commented queries
/notebooks     - EDA and modeling notebooks
/dashboard     - screenshot + link to published Tableau dashboard
/docs         - executive_summary.pdf
README.md     - project overview, tech stack, key findings, how to run it
```

Your README should lead with a 3-4 line summary and a screenshot of the dashboard — most people (recruiters included) will only look at the README, so it needs to sell the project in 10 seconds before they'd even click into your SQL folder.

4. Suggested Timeline (4 weeks, ~6-8 hrs/week)

Week	Focus
1	Postgres setup, data load, data-quality SQL, first 4-5 exploratory SQL queries
2	Finish SQL queries (window functions, CTEs), Python EDA + visualizations
3	Churn model + A/B test simulation
4	Dashboard build, executive summary, GitHub polish

5. How to talk about this in interviews

Prepare crisp answers to these — they *will* come up:

- "Walk me through your SQL query for X" — be able to explain your CTE/window function query line by line, not just that it "worked"
- "Why not just use pandas for everything?" — answer: SQL is the standard interface to production data warehouses at scale; doing the analysis at the SQL layer mirrors how a real analyst would query a data warehouse, and it's more scalable than pulling everything into memory
- "Your A/B test data is simulated — what would you do differently with real data?" — have a real answer ready: real experiment design would need pre-registration of the hypothesis, a proper randomization unit check, guardrail metrics (e.g., don't just optimize churn, watch for revenue impact), and a longer observation window
- "What would you do next with more time?" — good answers: cohort/survival analysis (time-to-churn, not just binary churn), causal inference methods (uplift modeling) instead of a simple simulated A/B test, or productionizing the pipeline with Airflow

Once you're ready, tell me and I can help with any specific piece in depth — e.g., I can write out the full SQL query set, review your notebook code, or help you draft the executive summary once you have real findings.